Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates

Воспроизведение

L. N. Smith, N. Topin, arXiv:1708.07120v3

Подготовил Башкин Михаил Александрович

Аннотация статьи

Описывается явление, названное авторами super-convergence: нейросети можно обучать на порядок быстрее, чем стандартными методами.

Ключевой элемент - обучение одним циклом learning rate с большим максимальным значением. Главное наблюдение: большой learning rate сам регуляризует обучение, поэтому остальные виды регуляризации нужно ослабить, чтобы сохранить их баланс.

Авторы также выводят упрощение метода Hessian-free для оценки оптимального learning rate.

Super-convergence показан на CIFAR-10/100, MNIST и ImageNet для ResNet, Wide ResNet, DenseNet и Inception. Чем меньше размеченных данных, тем больше выигрыш относительно стандартного обучения.

Тезисы статьи

Один цикл LR с большим максимумом ускоряет обучение на порядок: ResNet-56 - 92.4% за 10k итераций против 91.2% за 80k (§1, Fig. 1a)

LR range test показывает, возможен ли эффект: точность держится высокой вплоть до LR = 3 (§3, Fig. 2b)

Большой LR регуляризует: на участке LR 0.3–2 train loss растёт, test loss падает (§A.2, Fig. 8)

Регуляризацию нужно балансировать: при weight decay 1e-3 эффекта нет, при 1e-4…1e-6 есть (§A.6, Fig. 12b)

Выигрыш растёт при нехватке данных: +2.0 п.п. на 40k примеров, +9.2 п.п. на 10k (§5, Table 1)

Для ResNet-20: 90.4% (CLR) против 88.6% (ступенчатое расписание) (§5, Fig. 4b)

Оценка оптимального LR упрощённым Hessian-free - в этой работе не проверялась. (§4)

Проверяемые гипотезы

H1. 1cycle с пиком LR = 3 достигает точности ступенчатого расписания за в разы меньше итераций (E1, E2, E3)

H2. Сеть выдерживает LR на порядок больше обычного: в range test точность не обваливается (E0, E6)

H3a. Большой LR регуляризует: train loss растёт, test loss падает при LR 0.3–2 (E0)

H3b. При weight decay 1e-3 эффект пропадает, а при обычном LR - нет (E4, E4b)

H4. Чем меньше обучающих данных, тем больше выигрыш 1cycle (E5)

Эксперименты

ID	Расписание	Итерации	Weight decay	Особенность	Гипотеза
E0	linear 0 → 4	1 000	1e-4	оценка каждые 50 итераций	H2, H3a
E1	1cycle 0.1 → 3 → 0.001	5 000	1e-4	сиды 0, 1	H1
E2	step 0.35	20 000	1e-4	длинный baseline	H1
E3	step 0.35	5 000	1e-4	сиды 0, 1	H1
E4	1cycle, пик 3	5 000	1e-3, 1e-5, 1e-6		H3b
E4b	step 0.35	5 000	1e-3	контроль	H3b
E5	1cycle и step	5 000	1e-4	10 000 обучающих примеров	H4
E6	1cycle, пик 1 и 2	5 000	1e-4		H2

Допущения

Допущение	Зачем
Одна Tesla T4 в Colab вместо 8 GPU	Доступное железо
fp16 (autocast + GradScaler)	Тензорные ядра T4 ускоряют свёртки; GradScaler не даёт малым градиентам обнулиться
PyTorch вместо Caffe	Актуальный стек; поведение BN и SGD сверено с описанием в статье
Бюджеты 5k / 20k итераций (1:4, в статье 1:8)	Вся очередь укладывается примерно в 2 часа
Батч 1000	Как в статье; больший батч позволяет больший LR (Table 7)
BN momentum 0.05 для обоих расписаний	Это moving_average_fraction 0.95: при коротком обучении статистики BN должны успевать (§A.5)
Весь CIFAR-10 в памяти GPU, аугментации на GPU	Нет Dataloader – загрузка датасета не тормозит обучения
1–2 сида на эксперимент	Ограничение по времени; разницы < 0.5 п.п. считаем шумом
Range test 1000 итераций (в статье 5k–100k)	Экономия времени

Структура репозитория

`src/convergence_repro/`	устанавливаемый пакет с логикой
`models.py`	ResNet-20/56 для CIFAR (Tables 3-5 статьи)
`data.py`	CIFAR-10 на GPU, аугментации, кэш архива
`schedules.py`	расписания LR как функции номера итерации
`train.py`	один прогон: обучение, чекпоинты, resume, лог
`runs.py`	YAML-конфиги, список прогонов, --scale
`plots.py`	графики и сводная таблица из логов
`configs/E0–E6.yaml`	по файлу на эксперимент
`scripts/`	тонкие CLI над пакетом
`doctor.py, smoke.py`	проверка окружения и короткий прогон
`fetch_data.py`	загрузка CIFAR-10 с кэшем на Google Drive
`run_all.py, train.py`	очередь экспериментов / один прогон
`plots.py`	графики, таблица
`tests/`	pytest на CPU, 66 тестов
`colab.ipynb`	запуск в Colab
`README.md, REPORT.md`	инструкция и отчёт

Запуск

Colab

1. Загрузить colab.ipynb, выбрать T4
2. В ячейке Config указать REPO_URL
3. Runtime и Run all
4. После обрыва - снова запустить Run all: готовые прогоны пропускаются, прерванный продолжается с чекпоинта

```
# Локально (нужен GPU с поддержкой текущего torch)
uv sync
uv run python scripts/doctor.py --out runs
uv run python scripts/run_all.py --out runs
uv run python scripts/run_all.py --out runs --only E1,E3
uv run python scripts/plots.py --out runs --readme README.md

# Результаты
runs/<run_id>/log.json   # метрики, статус
runs/<run_id>/ckpt.pt    # чекпоинт для resume
runs/tb/                 # TensorBoard
runs/figures/, summary.md
```

Расписание LR

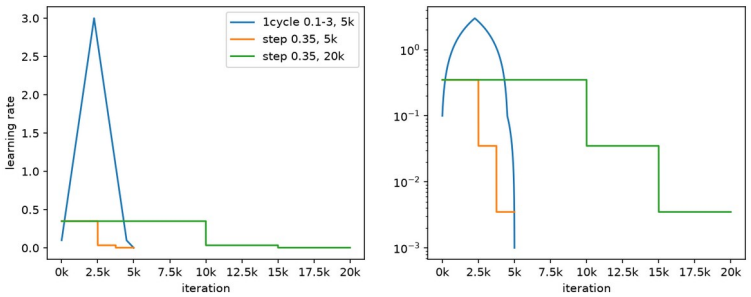
```
def onecycle(max_lr, total, base_lr=0.1, final_lr=1e-3, pct=0.45):
    e1 = pct * total - 1
    e2 = 2 * pct * total - 2
    e3 = total - 1
    def lr_at(it):
        if it <= e1:
            return base_lr + (max_lr - base_lr) * it / e1
        if it <= e2:
            return max_lr + (base_lr - max_lr) * (it - e1) / (e2 - e1)
        return base_lr + (final_lr - base_lr) * min(1.0, (it - e2) / (e3 - e2))
    return lr_at

# в цикле обучения
lr = lr_at(st["it"])
for g in opt.param_groups:
    g["lr"] = lr
```

Три линейные фазы: рост до пика, спад до 0.1, спад до 0.001.

Границы фаз совпадают с torch OneCycleLR(three_phase=True).

LR зависит только от номера итерации, состояния у расписания нет.



Шаг обучения в fp16

```
with torch.autocast(device.type, dtype=torch.float16, enabled=amp):
    logits = model(x)
    loss = loss_fn(logits, y)
opt.zero_grad(set_to_none=True)
scaler.scale(loss).backward()
scaler.step(opt)
scaler.update()
buf.append(torch.stack([loss.detach().float(),
                        (logits.argmax(1) == y).float().mean()]))
```

autocast - свёртки в fp16 на тензорных ядрах T4, loss в fp32.

GradScaler умножает loss перед backward, чтобы малые градиенты не обнулились в fp16; если градиенты получились inf, шаг пропускается.

buf -- метрики батча остаются на GPU.

Создание чекпоинта и продолжение после обрыва сессии

```
def _atomic(path, write):
    tmp = path.with_name(path.name + ".tmp")
    write(tmp)
    os.replace(tmp, path)          # файл подменяется одной операцией

obj = {
    "model": model.state_dict(), "opt": opt.state_dict(),
    "scaler": scaler.state_dict(), "state": dict(st),
    "perm": perm, "gen": gen.get_state(),
    "torch_rng": torch.get_rng_state(), "np_rng": np.random.get_state(),
    "cuda_rng": torch.cuda.get_rng_state_all(),
    "log": log, "config": cfg,
}
_atomic(ckpt_path, lambda p: torch.save(obj, p))
```

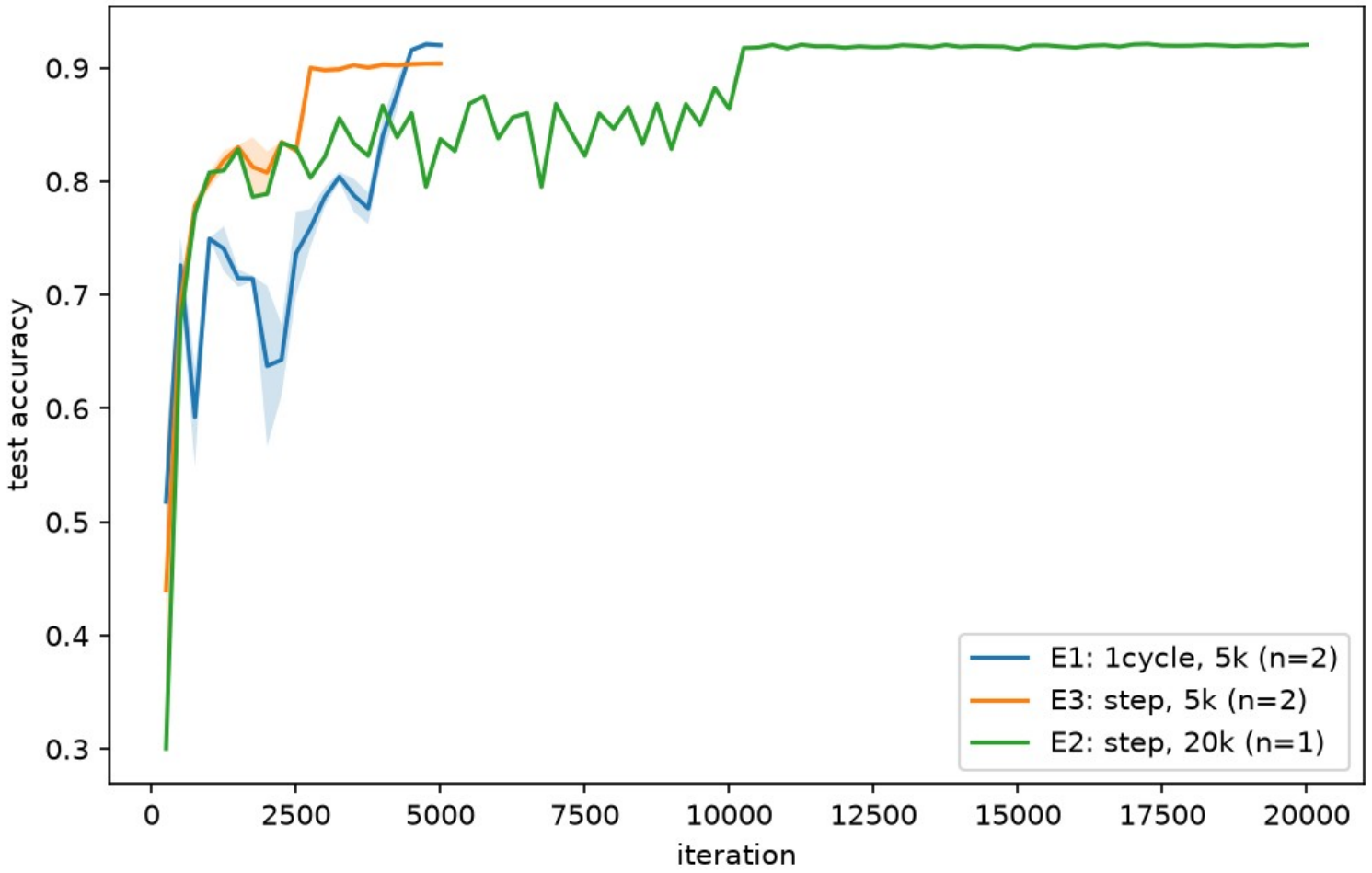
Обрыв Colab посреди записи оставляет на Drive старый целый файл, а не обрезанный.

Сохраняется всё для точного продолжения: веса, momentum, масштаб GradScaler, перестановка текущей эпохи и позиция в ней, все генераторы случайных чисел.

Аугментация батчем на GPU

```
def crop_flip(x, i, j, flip):
    n, c, h, w = x.shape
    p = F.pad(x, (PAD,) * 4, mode="reflect")           # 32 -> 40
    rows = (i[:, None] + torch.arange(h))[:, None, :, None]
    cols = (j[:, None] + torch.arange(w))[:, None, None, :]
    out = p[torch.arange(n)[:, None, None, None],
            torch.arange(c)[None, :, None, None], rows, cols]
    return torch.where(flip[:, None, None, None], out.flip(3), out)
```

H1. 1cycle достигает точности step в 4 раза быстрее

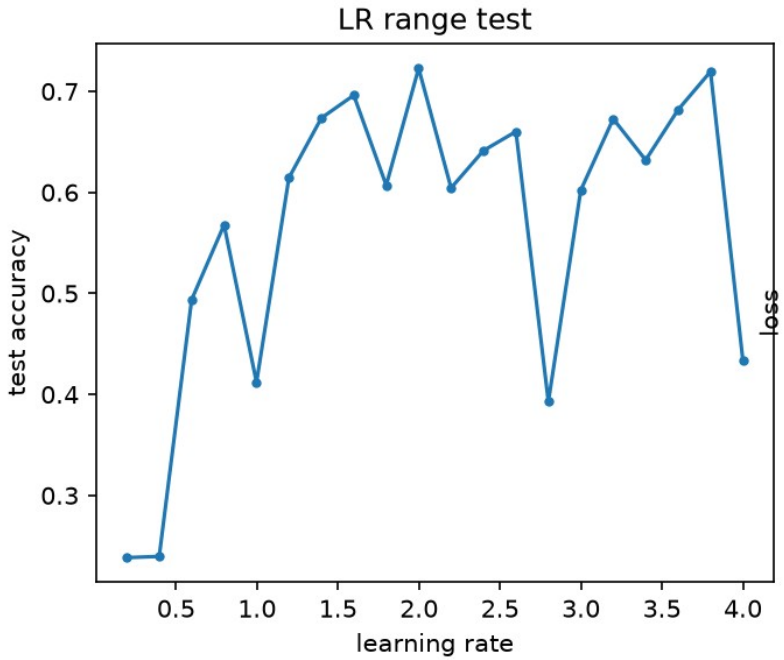


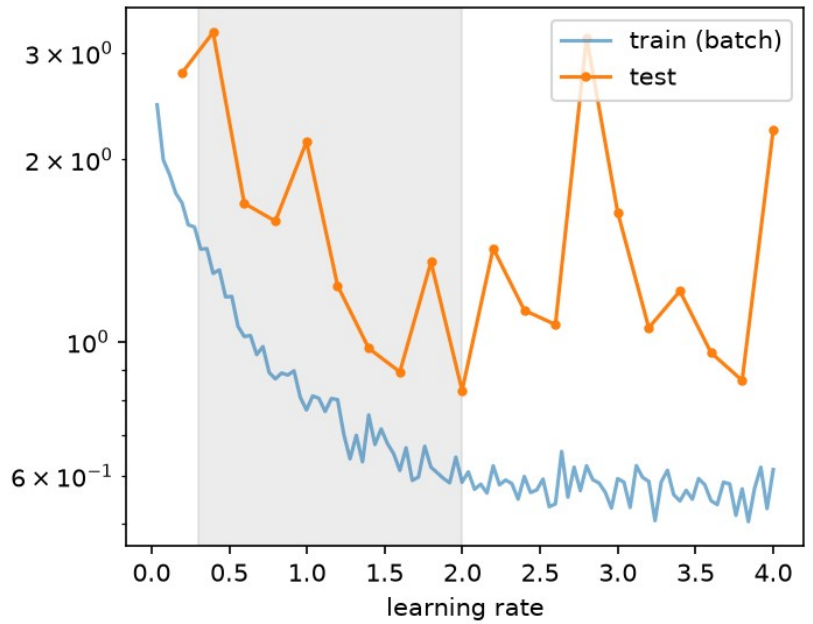
	Мы	Статья
1cycle	92.00 ± 0.25% (5k)	90.4% (~20k)
step, тот же бюджет	90.38 ± 0.42% (5k)	—
step, длинный	92.02% (20k)	88.6%

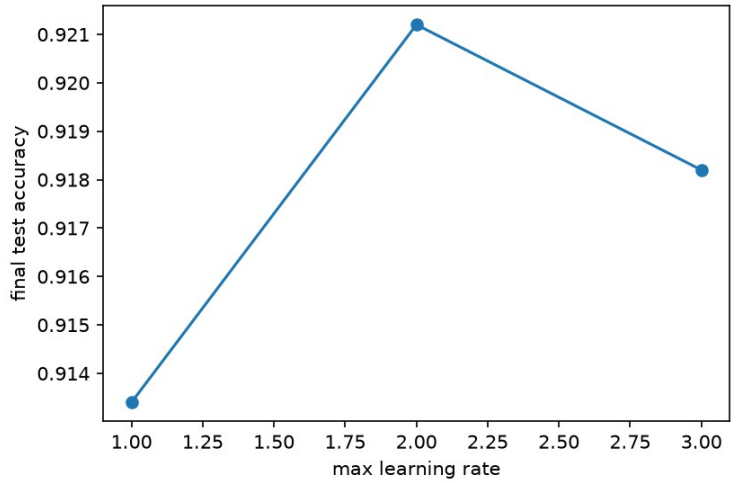
Подтверждается. Та же точность за четверть итераций, +1.6 п.п.

Смотреть: синяя кривая к 5k итераций выходит на тот же уровень, что зелёная к 20k. Провал синей в середине - момент пика LR = 3.

H2. Сеть выдерживает LR на порядок больше обычного





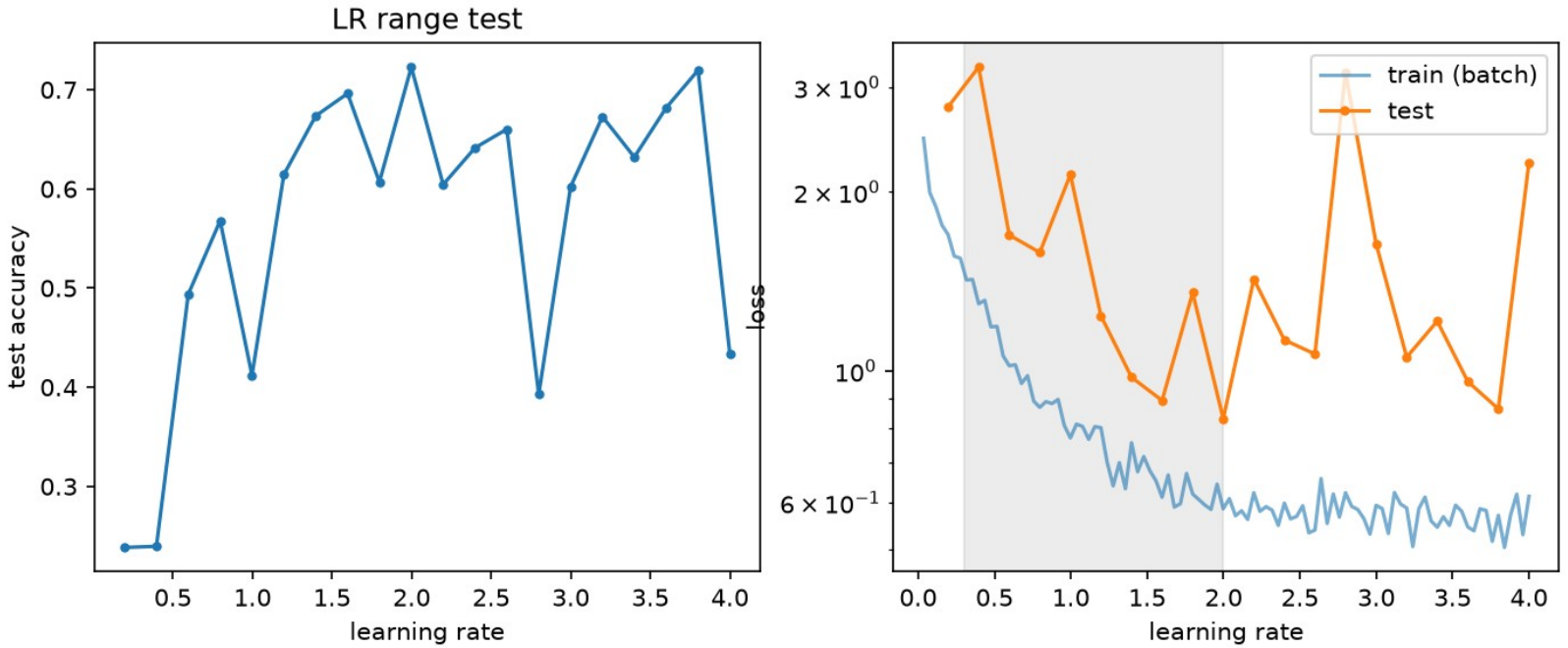


Смотреть: пик 2 и 3 дают примерно одно и то же, пик 1 - ниже.

Смотреть: левый график - после LR ≈ 1.2 нет обвала вплоть до LR = 4. Отдельные провалы - шум, важен общий уровень.

Подтверждается в целом. Точность range test держится на уровне 60–72% до LR = 4; пик 1 / 2 / 3 в 1cycle: 91.34 / 92.12 / 92.00%.

Н3а. Большой LR как регуляризатор

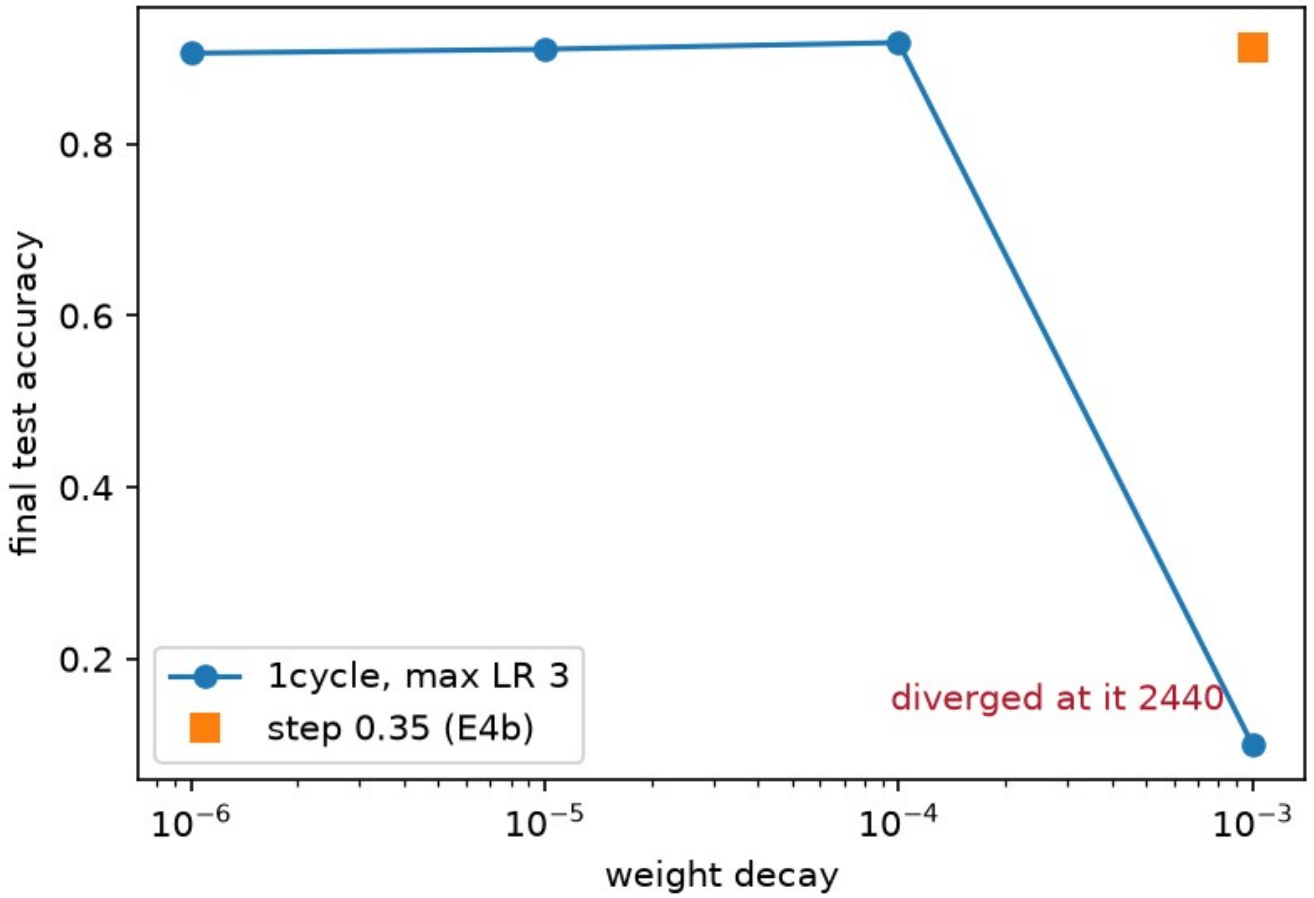


Смотреть: правый график, серая полоса LR 0.3–2. По статье здесь train loss должен расти, а test loss падать.

Не наблюдается. Train loss падает на всём диапазоне, test loss шумит без тенденции.

Возможная причина – недостаточная длина эксперимента (недообучение).

H3b. Регуляризацию нужно балансировать

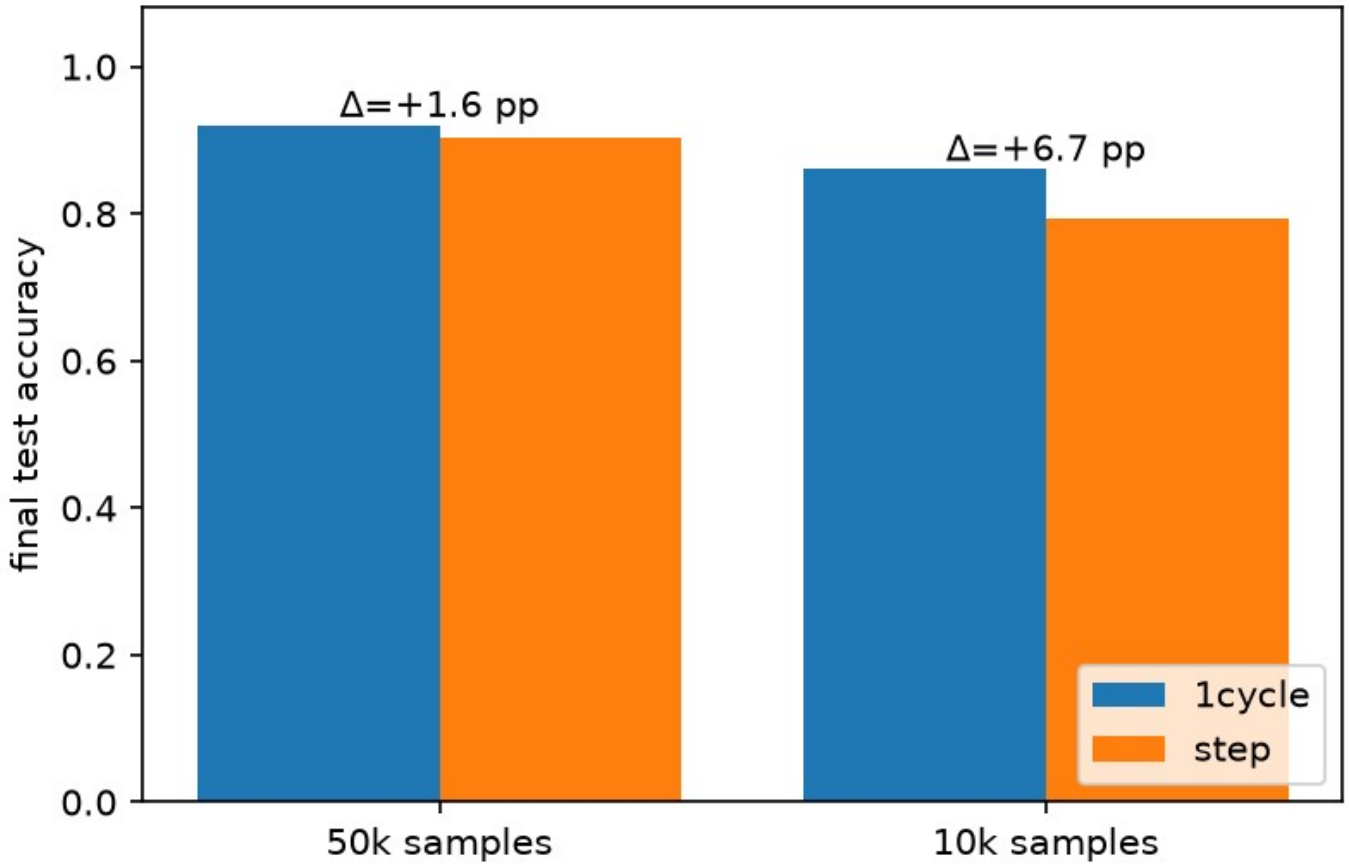


Weight decay	1cycle	step
1e-6	90.59%	—
1e-5	91.05%	—
1e-4	91.82%	—
1e-3	разошлось, итерация 2440	91.24%

Подтверждается. 1cycle с WD 1e-3 разошёлся почти в момент пика LR (итерация 2250). Step с тем же WD обучился нормально.

Смотреть: синяя линия ровная до 1e-4 и обрывается на 1e-3. Оранжевый квадрат (step) при том же 1e-3 - наверху.

H4. Выигрыш на малом количестве семплов



Примеров	1cycle	step	Δ, мы	Δ статья
50 000	92.00%	90.38%	+1.6	+1.2
10 000	86.03%	79.35%	+6.7	+9.2

Подтверждается. Разрыв вырос в четыре раза. Когда данных мало, сеть легче переобучается, и регуляризация от большого LR ценнее.

Смотреть: разница Δ над парами столбцов - 1.6 п.п. на полных данных и 6.7 п.п. на 10k.

Итог

Гипотеза	Результат	Вердикт
H1. 1cycle быстрее step	92.0% за 5k итераций = step за 20k	подтверждена
H2. LR на порядок больше	range test без обвала до LR = 4; пик 2-3 оптимален	в целом подтверждена
H3a. Большой LR регуляризует	эффект не виден: range test слишком короткий	не наблюдается
H3b. Баланс регуляризации	WD 1e-3 + 1cycle расходится, + step - нет	подтверждена
H4. Выигрыш при малых данных	+1.6 к +6.7 п.п. при 50k к 10k примеров	подтверждена

Главный результат статьи воспроизводится на одной T4 примерно за 2 часа: один цикл с большим LR даёт точность длинного ступенчатого расписания за четверть итераций при ослабленной остальной регуляризации.